Module 3: Imperative Languages—Control Flow

Topics

1. Expressions
2. Statements
3. Subprograms

I. Expressions

We begin our discussion of control flow in imperative languages with the lowest level of control flow—the expression level.

A. Expression Evaluation

Fortran first introduced nested expressions in high-level languages. When an infix expression contains more than one operator, one important question is how that expression should be parsed and, therefore, how it should be evaluated. In module 1, we emphasized the importance of designing a grammar that would produce the correct parse tree. Now we consider the rules of programming languages that specify the kind of parse tree that is required. Every language allows expressions to be fully parenthesized, including a pair of parentheses for each operator. When an expression is fully parenthesized, the parentheses parse the expression, thereby specifying the parse tree and how the expression is to be evaluated.
Most languages, however, allow parentheses to be omitted. When they are omitted, other rules are needed to specify how to correctly parse an expression. These rules are the precedence and associativity rules. Many books characterize these rules as defining the order of operations. Unfortunately, that characterization causes many

students to misunderstand the purpose of these rules. A common misunderstanding that arises by thinking that precedence defines the order of operations is illustrated by the following expression:

```
(x + y) - (a * b)
```

Some students conclude that the multiplication operator is evaluated before the addition operator because the former has higher precedence. That conclusion is incorrect because this expression is sufficiently parenthesized so that the precedence rules are irrelevant in this case.

The best way to characterize the purpose of the precedence and associativity rules in programming languages is to say their role is to determine the way the expression should be parsed when parentheses do not fully specify the correct parse tree.

Precedence Rules

Once you realize that nested expressions in programming languages are intended to mirror mathematical formulas, it will not surprise you that these expressions borrowed the precedence rules that are inherent in mathematical formulas. For example, in mathematics, when we write a formula such as $xy + ab$, we know it is interpreted as $(xy) + (ab)$ because multiplication has higher precedence than addition. What has *not* been copied from mathematics is the implied multiplication of mathematical formulas, in part, because single-letter variable naming is generally discouraged.

Languages vary greatly with regard to the number of levels of precedence, with a language such as Pascal having as few as 4 levels and languages such as C and C++ having as many as 15. Just as the number of precedence levels varies, so does the relative precedence of operators. Among the arithmetic, relational, and logical operators, however, there is a natural precedence order that most languages observe, ranking the arithmetic operators as the highest among the three groups and logical operators the lowest. Pascal is an exception to this general order. Consequently, in Pascal, writing the following expression without parentheses is an error:

```
(x > y) AND (y > z)
```

The parentheses are required because, in Pascal, the logical operators have higher precedence than the relational operators.

Associativity Rules

The associativity rules are needed in the absence of parentheses when there are two or more operators of equal precedence, as illustrated by the following expression containing integer constants:

```
4 - 1 - 1
```

Because subtraction is not associative, the result will be different depending on how the subexpressions are grouped. Grouping from left to right produces a value of 2, whereas a right-to-left association gives 4.

The four fundamental arithmetic operators associate left to right in almost all languages that define associativity rules. The one arithmetic operator that varies, in languages that have it, is exponentiation. In Visual Basic it is left associative, in Fortran, it is right associative, and in Ada it is nonassociative. By *nonassociative*, we mean that Ada requires parentheses in expressions that have adjacent exponentiation operators.

Subexpression Evaluation Order

One additional issue that programming languages must define is whether the left subexpression or the right is evaluated first, or whether this evaluation order is left unspecified. Upon initial reflection, it may not seem that this order matters, which is true in most cases. Consequently, most languages leave this order unspecified for most operators, to allow the optimizer phase of the compiler the freedom to make the optimal choice. As we shall discuss shortly, short-circuit logical operators, when present, are an exception. Java is also an exception. In Java, the left subexpression is always evaluated before the right.
Expressions in which this order matters are expressions that contain side effects. Side effects are caused in expressions either by assignment expressions or function calls that modify global variables or parameters or perform input/output.

Figure 3.1 provides an example of a C or C++ expression containing an assignment operator whose value depends upon the relative order of evaluation of the two subexpressions. By selecting one of two buttons that are provided, you can specify which order is used. After you step through the explanations when the Left-Before-Right button is selected, select the Right-Before-Left button to step through those explanations.

Figure 3.1
Evaluating Expressions with Side Effects

0:00 / 1:14

What you should have noticed is that the result of the expression will be different depending upon which order was followed. The results are different because this expression contains a variable whose value is changed by an assignment expression and the same variable is used again in another part of the expression.

Using a function call to a function that either modifies a parameter or a global variable can create a similar kind of expression. The following C++ program contains a function that modifies its parameter, which is also contained in another part of the expression.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int doubleParameter (int* parameter);

int main()
{
    int x = 10, y = 10, left, right;
   left = doubleParameter(&x) + (x / 2);
    right = (y / 2) + doubleParameter(&y);
    cout << left << " " << right << endl;
    return 0;
}

int doubleParameter (int* parameter)
{
    return (*parameter) *= 2;
}
```

When run with Microsoft Visual C++, this program produces 30 25 as its output. Other compilers could behave differently. When side effects are present, as they are in this case, the result is that addition is no longer a commutative operation, as this program illustrates.

Expressions without side effects are called *pure expressions*. It is difficult in imperative languages to guarantee that expressions will always be pure. Ada comes closest to achieving this goal. It has no assignment expressions. Function parameters must be in parameters, which prevents the kind of program written above in C++. The first Ada reference manual, written in 1979, stipulated that

functions were prohibited from modifying global variables to help ensure the purity of function calls. This stipulation was dropped by the time that version of Ada was finalized in 1983.
Pure expressions are extremely important in functional languages, so we will return to this idea in module 5.

B. Operators

Next, let's consider the various kinds of operators that make up expressions.

Arithmetic Operators

All imperative languages have the standard four arithmetic operations and use the symbols:

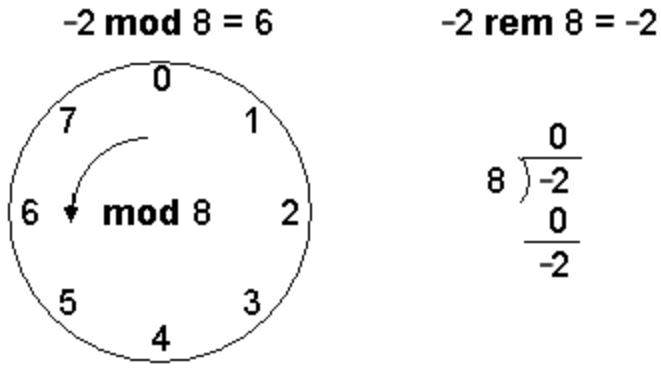
+
–
*
/

There is some disparity among languages, however, with integer division, the remainder operation, and exponentiation.

Pascal is unusual among languages, having an operator, DIV, for integer division. Using the / operator with two integer operands produces a real result, whereas the DIV operator produces integer division. In most other languages, one must type cast one of the integer operands to a floating-point type to avoid integer division, which otherwise would be the default.

The remainder operator is another varying feature. The C family uses the % symbol. Pascal uses the reserved word MOD. In most languages, Java being an exception, this operation is valid only with integer operands. Ada is unique in that it provides two operators, rem for remainder and mod for modulus, which produce different results with negative values. Figure 3.2 contains an example that illustrates this distinction.

Figure 3.2
Comparing the mod and rem Operators of Ada



We have already discussed some of the differences among languages with regard to the exponentiation operator when we described operator associativity. What we did not mention is that not all languages have an exponentiation operator. Fortran and Ada do have one and use the symbol **. The C family of languages, by contrast, has no operator defined for exponentiation, and relies instead on library functions.

Relational Operators

All languages have the standard six relational operators. What varies somewhat are the symbols used for equality and inequality.

- In Pascal and Ada, = is used for equality because = is not the symbol for assignment.
- In the C family, == is used because = is the symbol for assignment. (In module 1, we discussed the problem associated with mistakenly using = when == is meant in the C languages.)

The symbol for inequality varies more than the symbol for equality:

- In Pascal, <> is used.
- In Ada, /= is used.
- In the C family, != is used.

We summarize these facts in table 3.1 below.

Table 3.1
Equality and Inequality Symbols

	Equality	Inequality
Pascal	=	<>
Ada	=	/=
C family	==	!=

A problem is associated with relational operators in languages such as C and C++ that lack a true Boolean type. In such languages, expressions like:

```
1 <= x <= 10
```

are misinterpreted, as we discussed in module 2.

Another misinterpreted expression containing a relational operator is the following expression in C++:

```
"abc" < "def"
```

Rather than comparing the string values, the addresses of the two literals are compared instead, which is clearly not what is intended. The cause of this misinterpretation is the fact that strings in C++ are C-style strings whose type is char*. In module 2, we saw another example of how this fact can cause misinterpretation of the + operator.

Finally, a similar misinterpretation can occur in Java strings when the == operator is used rather than the equals method. The cause for the misinterpretation in Java is different, however. It is caused by the fact that Java does not overload the == operator for strings to perform deep comparison—comparison of values rather than references. Java disallows user-defined operator overloading as well—a topic we will discuss shortly.

Logical Operators

In addition to some variation in the symbols used, the other aspect of logical operators that tends to vary among languages is whether the operators short-circuit or not. Short-circuiting of logical operators is based on the following two logical identities that are true regardless of the value of *x*.

false AND $x \equiv$ false
true OR $x \equiv$ true

Logical expressions containing short-circuiting operators are unique in that the left subexpression must always be evaluated before the right, regardless of whether this rule applies to other expressions in a language. The reason is that when short-circuiting does occur, the right operand is never evaluated.

Ada has both short-circuit and non-short-circuit logical operators. The operators "and then" and "or else" short-circuit, whereas and and or do not short-circuit. Java has both kinds of logical operators also—&& and || short-circuit—& and | do not short-circuit. For clarity, we have summarized these in the table below.

Table 3.2
Short-Circuit and Non-Short-Circuit Operators
in Ada and Java

	Short-Circuit Operators	**Non-Short-Circuit Operators**
Ada	and then or else	and or
Java	&& ||	& |

In C and C++, the bitwise operators cannot be used reliably as non-short-circuiting operators because those languages treat 0 as false and nonzero values as true. Consequently, the following attempt will fail:

```
10 & 01 = 00
```

Two values that are regarded as true ANDED together should produce a true result but clearly do not.

Conditional Expression Operator

The languages in the C family have a conditional expression operator, but Pascal and Ada do not. It is an unusual operator in two ways. First, it is a ternary operator, meaning it has three operands. Second, it involves two symbols, the ? and the : symbols. The only case in C and C++ where an if statement cannot be used in place of this operator is with macros, as illustrated by the following example.

```
#define MAX(x,y) x > y ? x : y
```

Because this operator is rather cryptic, one might reasonably have argued for dropping it from Java, which does not permit macros, but the Java designers clearly believed otherwise.

Lest anyone doubt that overuse of this operator can lead to hard-to-read programs, consider the following use of nested conditional expression operators to determine the largest of three values:

```
x > y ? x > z ? x : z : y > z ? y : z
```

The lack of parentheses adds to the difficulty in interpreting this expression and requires that one know that it is a right-associative operator.

Bitwise Operators

Including bitwise operators in a language is a low-cost addition because every machine has hardware instructions to perform such operations. C, which was intended as a language to replace assembly language, understandably included them. All the languages in the C family kept them. There are six bitwise operators in C, as shown in the following table:

Table 3.3
The Bitwise Operators of C

Bit-Wise Operator	Meaning
&	and
|	or
^	exclusive or

<<	left shift
>>	right shift
~	one's complement

Because Java removed unsigned types, it added a seventh operator:

```
>>>
```

In Java, the >> operator performs an arithmetic right shift, which is what occurs in C when signed types are used. The >>> operator performs a logical right shift, which is what occurs in C with unsigned data.

The Comma Operator

Although a comma is a punctuation symbol in most programming languages, in C and C++, it is an operator when it appears in certain contexts. It is most commonly used in for loops, in which two subscripts are needed. For example:

```
for (int i = 0, j = n - 1; i < n; i++, j--)
    ...
```

It is most noteworthy, however, because of the potential problems it can create. Consider the following assignment:

```
x = a * b,
     + c;
```

Beginners occasionally misunderstand the role of a comma to be a symbol required when a statement is continued on the next line. Such a misunderstanding might lead one to write an assignment like the one above. Unfortunately, the above statement is not a compilation error; it is a logic error because the value of the expression is only a * b. The comma is viewed as the comma operator and the + as a unary plus.

The Java developers wisely chose to exclude this operator.

Assignment Operators

In languages that regard the assignment symbol as an operator, which includes all languages in the C family, assignments are expressions. By contrast, in Pascal and Ada, assignments are statements. When assignments are expressions, they can be nested wherever an expression is permitted that enables statements such as the following commonly used C loop:

```
while ((ch = getchar()) != '\n')
    ...
```

By nesting the assignment to the call to the function that reads in the next character, one is able to avoid duplicating that call—first in a priming read before the loop and subsequently at the end of the loop body.

Another benefit of assignments being expressions is that assignments can be chained together, as illustrated by the following assignment:

```
x = y = 1;
```

One must also realize that the assignment operator is right associative to understand the way the above statement is interpreted.

Making assignments expressions does have two drawbacks:

1. It contributes to the misinterpretation of statements in which the assignment operator is mistakenly used where the equality operator was intended. We discussed that issue in module 1.

2. It introduces side effects in expressions, an issue we discussed earlier in this module, that can lead to different expression values depending upon the order in which the subexpressions are evaluated.

Although Pascal and Ada have only a single assignment symbol, :=, languages in the C family have a whole collection of assignment operators. In addition to the simple assignment operator, any of the binary arithmetic or bitwise operators can be

combined with the assignment operator to produce operators like +=. Furthermore, these languages also have increment and decrement operators, which are also assignment operators that can be used either in prefix or postfix.
The numerous assignment operators are an example of feature multiplicity—a characteristic of language features that we introduced in module 1. Specifically, in the C family of languages, there are, for example, four different ways to increment a variable such as *x*. These four possibilities are:

```
x = x + 1;
x += 1;
x++;
++x;
```

Providing so many different ways to accomplish the same task can lead to programs that might be difficult to read. In this case, the three operators +, +=, and ++ have different degrees of applicability. Specifically, the + operator is most general and ++ the least general. So, if programmers adopt the style of using the least general operator that can be applied in a given case, the readability of programs can be enhanced. Choosing the least general operator does not, however, help you decide whether to use the increment operator in prefix or postfix when the meanings do not differ because neither is more applicable than the other, so some convention is needed to establish consistency.

Finally, we consider type checking of assignments. In module 2, we mentioned that in C and C++, unexpected results can occur when variables of a larger type are assigned to variables of a smaller type—assigning a variable of type long to one of type short, for example. Java requires explicit type casts when narrowing assignments are performed to call attention to the potential consequences of such assignments.

C. Operator Overloading

Most languages overload some operators—the arithmetic operators being the most commonly overloaded. Arithmetic operators are generally defined to perform integer operations when applied to integers and to perform floating-point

operations when applied to floating-point numbers. What is less universal is whether languages allow user-defined overloaded operators. Both C++ and Ada allow such overloading. Java does not.
Proponents argue that allowing user-defined overloading enables data types that appear to be built-in types to be added to a language. Detractors claim the misuse of this language feature can lead to programs that are hard to read.

II. Statements

Now we consider the next higher syntactic level of control flow—the statement level.

A. Fully Bracketed Statement Syntax

We begin with a general issue related to control-statement syntax, which is the presence or absence of a fully bracketed syntax—a syntax that provides both an opening and a corresponding closing reserved word for each statement. Many newer languages have such a syntax. Ada employs this syntax, as does Fortran, beginning with Fortran 77. The languages in the C family do not.

A fully bracketed syntax has three benefits. First, requiring an end if terminator on if statements eliminates the dangling else problem that we discussed in module 1 in conjunction with our discussion of ambiguous grammars. Consider parallel if statements in C++ and Ada to see why in C++ ambiguity is possible, but in Ada, it is not.

Move your mouse over the highlighted parts of the program segments to see the explanations of those parts.

Table 3.4
Comparison of C++ and Ada if Statements—
When Another if Is Nested in the if Part

C++ Version	Ada Version

`if (x != 0)`
`if (x > 0)`
`cout << "x is positive";`
`else`
`cout << "x is zero";`
	`if x /= 0 then`
	`if x > 0 then`
	`put("x is positive");`
	`end if;`
	`else`
	`put("x is zero")`
	`end if;`

The second benefit of a fully bracketed syntax is that because all statements are explicitly terminated, the inconsistency of when compound statements are required is eliminated. Consider parallel if statements in C++ and Ada containing a single statement. Notice that when a second statement is added, the C++ if statement must be modified to contain a compound statement. No such change is needed in Ada.

Table 3.5
Comparison of C++ and Ada if Statements—
When Two or More Statements Are Nested in an if Statement

C++ Version	Ada Version

`if (x < 0)`
`{`
`x = -x;`
`cout << x;`
`}`
	`if x < 0 then`
	`x = -x;`
	`put(x);`
	`end if;`

The third benefit to a fully bracketed syntax is that because each kind of statement has a unique closing reserved word, as Ada does, error recovery can be improved. Consider a parallel missing closing delimiter in C++ and a corresponding error in Ada.

Move your mouse over the highlighted parts of the program segments to see the explanations of those parts.

Table 3.6
Comparison of C++ and Ada Statements—
Error Recovery

C++ Version	Ada Version

`while (x != 0)`
`{`
`  cin >> x;`
`  if (x < 0)`
`  {`
`  x = -x;`
`  cout << x;`
`}`
	`while x /= 0 loop`
	`  get(x);`
	`  if x < 0 then`
	`  x = -x;`
	`  put(x);`
	`end loop;`

B. Selection Statements

Next, we consider the first of two fundamental categories of control statements—selection statements. Selection statements allow the control flow of a program to be broken into multiple paths. There are two fundamental kinds of selection statements. The first—the if statement—is the more general of the two. The second is more restrictive. It is called a case statement in some languages and a switch in others.

Let's begin with the if statement. We have already considered one fundamental variation in the syntax of this statement, which is whether it requires a closing reserved word. Another variation is whether the statement syntactically is restricted to at most two paths, or whether it allows the control flow to be separated into any number of paths.

The if statement of the C family of languages and the if statement in Pascal allow at most two paths. To achieve the effect of more than two paths, nested if statements are needed. By contrast, Ada's if statement allows any number of paths because it

allows any number of elsif clauses. Compare a code segment that determines a letter grade from a numeric grade using C++ with a comparable code segment in Ada.

Table 3.7

Comparison of C++ and Ada Selection Statements

C++ Version	Ada Version

`if (grade >= 90`
`  cout << "A";`
`else`
`  if (grade >= 80)`
`  cout << "B";`
`  else`
`  if (grade >= 70)`
`  cout << "C";`
`  else`
`  if (grade >= 60)`
`  cout << "D";`
`  else`
`  cout << "F";`
	`if grade >= 90 then`
	`  put("A");`
	`elsif grade >= 80 then`
	`  put("B");`
	`elsif grade >= 70 then`
	`  put("C");`
	`elsif grade >= 60 then`
	`  put("D");`
	`else`
	`  put("F");`
	`end if;`

Proper indentation generally reflects nesting, which is what we have done in both cases. Notice what occurs in the C++ code, however. The last cases become very deeply indented. In languages with two-way if statements, programmers may opt to indent deeply nested if statements the way we have shown them indented in Ada because that indentation better reflects the meaning of the statement.

Next, let's consider the other kind of selection statement. We have already observed that it is a less general statement than the if. Consequently, languages would not restrict a programmer's problem-solving ability by including only an if statement. Including both kinds of statements also introduces feature multiplicity, but because one statement is more restrictive than the other, with good programming style guidelines, such multiplicity need not compromise readability.

Regardless of whether we are considering the switch statement used by the C family of languages or the case statement of Pascal and Ada, the restrictions on these statements are much the same. The type of the expression that is used for selection must be an integral type, or in Ada's terminology, a *discrete type*. The expressions that define the cases must be constant expressions—meaning that their values must be able to be computed at compile time. Unlike the if statement, these statements do not use logical expressions for path selection.

Comparing the switch statement to Ada's case statement, we find some important differences. The need for break statements at the end of the cases inside switch statements is contrary to the way we expect true structured statements to behave. The switch statement is actually more similar to Fortran's computed goto statement than to the case statements of either Pascal or Ada. Because forgetting to place a break at the end of each case is a common programming error, C# requires each nonempty case to end with a statement that transfers control—usually a break or a return statement.

Another difference is in the ease with which multiple values can be specified for a particular case. With the switch statement, each value must be enumerated. In the Ada case statement, a rich syntax is provided that allows a collection of values to be specified in a very compact way.

C. Iterative Statements

The other major category of control statements is iterative statements. Iterative statements differ fundamentally from selection statements in that they produce backward control flow, which is impossible with any number of selection statements.

But like the selection statements, iterative statements can be divided into two similar groups. Table 3.8 shows those groups for both categories of statements in Ada.

Table 3.8
The Control Statements of Ada

	Discrete Values	Logical Condition
Selection	case	if
Iteration	for	while

In much the same way as the Ada case statement is restricted to selecting among a collection of values of a discrete type, the Ada for statement is restricted to iterating across a range of discrete values. Neither the case or for statements involve logical conditions, but both the if and while statements do. The for statement is guaranteed to never produce an infinite loop because the loop control variable cannot be modified within the loop. The same, of course, cannot be said of the while loop.

We have confined the preceding discussion to Ada because although the C family of languages has a for loop, it is not more restrictive than the while statement. In the C family, any for statement could be changed to a while and vice versa. This kind of feature multiplicity is less desirable because it becomes more difficult to clearly establish a good set of style rules concerning when to use either statement.

The syntax of the C family's for statement allows some clever, although perhaps obtuse uses. Consider the following C for statement, which performs a string copy that uses an assignment expression as its termination condition.

Move your mouse over the highlighted parts of the statement to see the explanations of those parts.

```
for (i = 0; s[i] = t[i]; i++)
    ;
```

Such statements illustrate the expressive power of the syntax, but also the difficulty of reading the statements that result.

Both C# and version 5.0 of Java have included variations of a for statement that are more restrictive than the while statement. (In C#, it is called a foreach statement.) These statements are useful for iterating across data structures for which an iterator is provided. In Java, this statement can be used, for example, to iterate across an array, as shown below:

```
int[] array = {1, 2, 3, 4, 5};
for (int value: array.values())
    System.out.println(value);
```

These iterators are read-only, so a similar construct cannot be used to assign values to every element of an array.

One other aspect of iterative statements that varies among languages is whether the language provides loops that allow the condition that governs whether the loop will continue or stop to be placed either at the beginning or the end of the loop. The languages in the C family do provide such a choice with the while statement containing the condition at the beginning and do-while statement containing the condition at the end. Loop statements that have the condition before the loop body may never execute, whereas those with the condition placed at the end are guaranteed to execute at least once. Pascal has both a while and a repeat-until statement, giving the programmer the choice of placing the condition at the beginning or the end. The Pascal while statement differs from the repeat-until, in that the condition for the while statement is a continuation condition, whereas the condition in the repeat-until is a termination condition. Ada does not provide any special syntax comparable to a do-while or repeat-until statement.

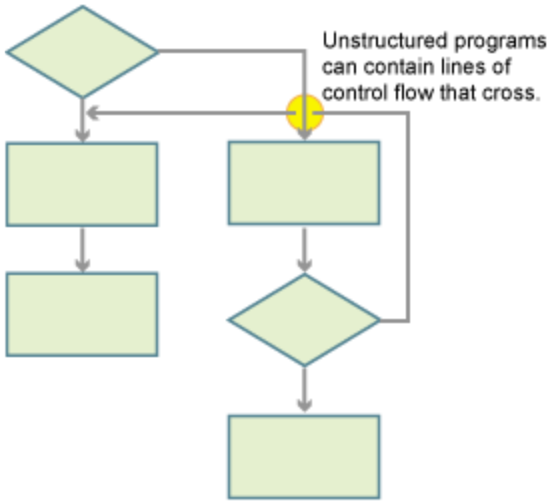
One final decision language designers must make in conjunction with iterative statements is whether multiple exits are permitted. Pascal, which was developed when structured programming was becoming widely accepted, took the approach of prohibiting multiple entries and exits. Although it is universally agreed that multiple entries are undesirable, the view on multiple exits has changed since that time. In the C family of languages, a break statement is used to exit from the middle of a loop. In Ada, the statement is called an exit statement. Java, C#, and Ada all allow a loop name to be specified so that an exit can be made from inside nested loops.

D. Structured Programming

Although it is taken for granted today, the introduction of a nested-statement syntax that enabled structured programs to be written was one of the most important milestones in programming language design. In assembly language and early versions of Fortran, programmers had to rely on *goto* statements to implement program control flow.

When programming in assembly and early Fortran, the design diagram used most often was the flowchart. With unstructured languages, the control flow of a program could be very convoluted, so a pictorial representation—the flowchart—helped clarify an otherwise potentially complicated algorithm. The flowchart of an unstructured program can be complicated because the lines of control flow can cross each other, as illustrated in figure 3.3

Figure 3.3
Flowchart of an Unstructured Program



The term *spaghetti code* was often used to describe programs with highly complicated flowcharts. This metaphor was used to compare the lines of control flow of such programs with the strands of spaghetti on a plate.

Once the benefit of structured programming was clearly understood and programmers refrained from using *goto* statements in their programs, the control flow of programs was greatly simplified, so flowcharts became unnecessary.

It is curious that in spite of the fact that it has long been recognized that using *goto*

statements is undesirable, Java is one of the few languages to have eliminated them entirely.

III. Subprograms

Finally, we consider the highest syntactic level of control flow—the subprogram level.

A. Parameter Passing and Return Values

One of the most important features of subprogram design is what mechanism a language provides for exchanging information between the calling subprogram and the called subprogram. We concentrate on two techniques—parameters and return values. Modifying global variables is another technique that can be used in languages that have global variables, but it is generally regarded as a poor design choice. There are really no design issues other than scope, which we have already discussed in module 2, that influence the ability to access global variables. Subprograms that are methods can access instance variables, but we defer specific discussion of methods to module 4, which focuses on object-orientation.

The way parameters are passed is a feature of programming languages that varies more than perhaps any other. One of the reasons for so much variation is that there are interacting and competing issues that any language designer must consider, so no solution is really a perfect one.

The two issues that interact are the semantic roles of parameters and implementation details. Semantically, a programmer must decide which of the following three roles the parameter will play:

1. to send data from the caller to the called subprogram, referred to as *in*
2. to send data from the called subprogram back to the caller, referred to as *out*
3. to send data in both directions, referred to as *in-out*

The implementation details involve how the parameters are passed:

- by value

- by result
- by reference
- by constant reference
- by value-result
- by name

What complicates matters is that these issues are not independent of one another. To illustrate this point, let's consider the relationship between three semantic roles of parameters and two of the implementations. Table 3.9 shows this relationship. The checkmarks mean that the role is supported by that implementation. An x indicates that it is not.

Table 3.9
Comparing Parameter Semantics and Passing Technique

	Value	Reference
in	√	√
in-out	x	√
out	x	√

Passing-by-value, which involves copying the actual parameter to the formal parameter, can be used only to send data into the subprogram, whereas passing by reference can be used for all three.

Because passing-by-reference appears to be more general, one might conclude it should be the only technique needed. Unfortunately, there is a competing issue that one also must consider when judging the generality of the parameter-passing technique shown in table 3.10.

Table 3.10
Comparing the Actual Parameter and the Passing Technique

Actual Parameter	Value	Reference
constant	√	x
variable	√	√
expression	√	x

This table indicates that when pass-by-value is used, the actual parameter can be a constant, variable, or expression, but with pass-by-reference, the actual parameter must only be a variable. When pass-by-reference is used, the rule for what is allowed as an actual parameter is the same rule as what can appear on the left side of an assignment.

If one considers this measure, call-by-value is more general. Call-by-value also has the benefit that it does not create aliasing as does call-by-reference. We will shortly see an example of the consequences of aliasing.

Because there is so much variation among languages, we have chosen seven languages to discuss—no two using exactly the same techniques.

Pascal

Pascal supports both call-by-value and call-by-reference, allowing the programmer to make the choice. What is perhaps most interesting about its implementation is the syntax for reference parameters, which is to require that they be labeled with the reserved word VAR. The choice of this syntax was, no doubt, to underscore the fact that when reference parameters were used, the corresponding actual parameters were required to be variables.

Fortran

Most early versions of Fortran used pass-by-reference. Unfortunately, Fortran has no restriction on the syntax of the actual parameters, as does Pascal. Consequently, it was possible to write Fortran programs that modified constants, which is a serious error. If a constant were passed as the actual parameter and the corresponding formal parameter's value were changed in the subprogram, the result could be that the constant's value would thereafter contain the new value.

Later versions of Fortran tend to use pass-by-value-result, which is sometimes also called *copy-in copy-out* because that name describes exactly how it is implemented. At the point of the call, the value of the actual parameter is copied to the formal parameter, and upon return, the formal is copied back to the actual. With this implementation, overwriting constants is easier to prevent.

We will return to comparing pass-by-reference and pass-by-value-result when we discuss Ada 83.

ALGOL

We have seldom discussed ALGOL in our previous topics concerning imperative languages, but it is noteworthy in this case because one of its parameter-passing techniques is unusual. It uses the familiar pass-by-value but also another technique called *pass-by-name*.

Pass-by-name textually substitutes the actual parameter for the formal parameter. In most cases, pass-by-name behaves exactly like pass-by-reference, but there are some peculiar situations where it behaves differently, illustrated by the following ALGOL procedure.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure OUTER;
    integer INDEX;
    integer array NUMBERS[1:2];
    procedure INNER(PARAMETER)
        integer PARAMETER;
        begin
        INDEX := 1;
        PARAMETER := 10;
        INDEX := 2;
        PARAMETER := 20
        end;
    begin
        INNER(NUMBERS[INDEX])
    end;
```

If you studied the highlighted statements in the above procedure, you saw that with call-by-name, formal parameters could refer to different variables at different times during the execution of the procedure. Implementing call-by-name is expensive

because this textual substitution must be done at run time, so code must be inserted at every point where a formal parameter is referenced that determines to what memory location that formal parameter currently refers.
Although no current widely used language uses call-by-name, C and C++ use compile-time textual substitution with macros, which we will discuss shortly.

C /C++

The parameter-passing technique used by C and C++ is call-by-value. In C, pointers are used to achieve passing by reference. Because array names are constant pointers, arrays are effectively passed by reference. As we illustrated with some examples in module 2, C++ added references, which provides a syntactically simpler technique for passing by reference.

C++ has two techniques for achieving in semantics. The first is to use call-by-value. The other is to use call-by-constant-reference. The latter technique avoids having to copy the actual to the formal parameter, which for large objects may be an efficiency benefit. Strictly speaking, using constant reference is slightly different from using call-by-value because modifying the value of the formal parameter is prohibited, which is not true when passing by value. The function prototype of a function that uses constant reference is shown below:

```
void f(const LargeObject& object);
```

The const modifier can also be used with pointer parameters. Unlike references, with pointer parameters, there are two things that we might wish to prohibit from changing:

1. the value of the pointer
2. the value the pointer points to

To illustrate these two possibilities, consider the following two C++ functions.

Move your mouse over the highlighted parts of the functions to see the explanations of those parts.

```
void f1(int* const p)
{
    p = new int;
    *p = 5;
}
void f2(const int* p)
{
    p = new int;
    *p = 5;
}
```

In much the same way that the references of C++ can be used to pass parameters by reference, they can also be used to return references to achieve what we might call *return-by-reference*. The following C++ program illustrates an integer wrapper class that contains two member functions that use return-by-reference.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

class Int
{
public:
    Int(int value)
    {
        this->value = value;
    }
    int& get()
    {
        return value;
    }
    friend ostream& operator<<(ostream& out, const Int& anInt)
    {
        return out << anInt.value;
    }
private:
    int value;
};

int main()
{
    Int i(5);
    i.get() = 6;
    cout << i << endl;
    return 0;
}
```

The ability to return references makes it possible for the C++ string class to overload the subscripting operator so that a subscripted string can be placed on the left side of an assignment to modify one character in a string.

Java

The parameter-passing rules in Java are really quite simple—everything is passed by value. It is often mistakenly written that Java objects are passed by reference, but such statements are really a misunderstanding of pass-by-reference. It is because all objects in Java are allocated on the heap and all object names contain references that such statements are made. When Java objects are passed as parameters, a copy of the reference is made, which is passing by value.

When pointer parameters are used in C++, we saw that const can be applied to either the pointer or the value being pointed to. In much the same way, the meaning of *in-out* semantics might be interpreted in two ways when reference parameters are used. It might mean either:

1. a change to a parameter's value will be preserved upon return to the caller

   or

2. a change to a parameter's reference will be preserved upon return to the caller

To illustrate the difference, consider the following Java program. Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
import java.awt.*;

class JavaParameters
{
    public static void main(String[] args)
    {
        Point point1 = new Point(100,100),
            point2 = new Point(200, 200);
        zeroPoints(point1, point2);
        System.out.println("point1 = " + point1 +
            " point2 = " + point2);
    }
    private static void zeroPoints(Point first, Point second)
    {
        first.x = first.y = 0;
        second = new Point(0, 0);
    }
}
```

Java object parameters can be viewed as *in-out* with regard to their values, but not with regard to their references.

Primitives are allocated on the stack, so when primitive parameters are used they cannot even be *in-out* with regard to their values. Consequently, it is impossible to write a swap method in Java that swaps two ints, for example.

C#

By contrast to Java, C# does allow pass-by-reference. It does so by adding the reserved word ref to the parameter declaration. So, if the Java program we just examined were rewritten in C#, and the parameter second were labeled as a ref parameter, both points would be zeroed out when the program was run.

C# also allows parameters to be labeled with out to designate *out* semantics. In that case, the parameter need not be initialized prior to the method call.

Ada

Although Ada predates C# by about two decades, the two have similar approaches to parameter passing. In Ada, procedure parameters are designated with one of three reserved words—in, in out, or out—that correspond to the three semantic roles of parameters. Omitting any of the three reserved words will cause a default to the in mode. In Ada 83, in parameters could not be written to, and out parameters could not be read from. In Ada 95, the latter restriction was dropped.

Ada's design philosophy was that the programmer should specify the semantics, and the compiler should decide on the appropriate implementation that supports it. We have already explained how some parameter-passing techniques such as call-by-value can accommodate only certain semantic roles, whereas others can accommodate any of the three. In particular, both call-by-reference and call-by-value-result can be used to implement *in-out* semantics.

When we were discussing the technique of constant reference in C++, we noted that it could be used in place of call-by-value because it might be more efficient given that it avoids making a copy. The efficiency tradeoffs between techniques involving copying compared to those that don't are actually more complicated.

In Ada 83, the compiler is free to choose whether to use call-by-reference or call-by-value-result when array parameters are designated as in out. The reasoning is that when the array is small but the number of array references is large, making a copy is preferable. Keep in mind that when call-by-reference is used, referring to array elements is less efficient because it requires an additional level of indirection. On the other hand, if the array is large and the number of references small, it is better to avoid copying the array. In theory, Ada compilers could have performed an efficiency analysis and chosen the more efficient implementation for that program. In practice, it is unlikely that any did.

Although allowing the compiler the freedom to choose the more efficient parameter-passing technique seemed like a good design decision, one unfortunate result was that certain kinds of programs—those that involved an undesirable kind of aliasing—would behave differently depending upon which technique was used.

Figure 3.4 provides an example of an Ada program containing this kind of aliasing. By selecting one of the two radio buttons provided, you can specify which

parameter-passing technique is used.

Figure 3.4
An Ada 83 Program that Behaves Differently,
Depending upon the Chosen Parameter-Passing Technique

0:00 / 1:54

Ada 83 declared such programs illegal, but compilers did not detect this illegality nor did they produce a compilation error. In recognition of the limited practical value of this design decision and the accompanying problems, in Ada 95, call-by-reference is used exclusively in such programs.

Unlike the C family of languages, Ada distinguishes between functions and procedures. Because function calls are expressions whereas procedure calls are statements, Ada places additional restrictions on function parameters, as we mentioned earlier. In Ada functions, the parameters can only be in parameters to minimize side effects.

B. Other Parameter Issues

There are several other characteristics of parameters aside from the way they are passed that we consider next.

Default Parameters

The first of these characteristics is whether a language allows default parameters. Both C++ and Ada allow such parameters, but Java does not. In Ada, default parameters are allowed only with in parameters. Default parameters can be very useful with constructors, as the following C++ example illustrates.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
class Time
{
public:
    Time(int hours, int minutes = 0, int seconds = 0)
    {
        this->hours = hours;
        this->minutes = minutes;
        this->seconds = seconds;
    }
private:
    int hours, minutes, seconds;
};

int main()
{
    Time noon(12);
    Time eightTwenty(8, 20);
    Time oneOneOne(1, 1, 1);
    return 0;
}
```

To write a class in Java equivalent to the one above would require including three separate constructors.

Default parameters can interact with another feature that we will discuss in greater detail shortly—subprogram overloading. As an illustration of this interaction, suppose an Ada program had two overloaded procedures with the following specifications:

```
procedure Some_Procedure(Formal_1: in Integer;
    Formal_2: in Boolean := False);

procedure Some_Procedure(Formal_1: in Integer;
    Formal_2: in Integer := 0);
```

Next, suppose the program included the following procedure call:

```
Some_Procedure(1);
```

Clearly, such calls are ambiguous because there is insufficient information to determine which one is intended. Compilers can detect such ambiguities, and Ada compilers will generate a compilation error in such cases. Such feature interactions are far less serious than the ones that result in a program being accepted by a compiler but misinterpreted.

Parameter Association

In most languages, the association between the actual and formal parameters is positional association—the first actual matches the first formal and so on. In Ada, the programmer can choose whether to use positional association or named association. To illustrate these two possibilities in Ada, consider the following procedure specification:

```
procedure Another_Procedure(Formal_1: in Integer;
    Formal_2: in out Boolean; Formal_3: out Float);
```

Using positional association, the procedure call would be written as follows:

```
Another_Procedure(Actual_1, Actual_2, Actual_3);
```

Using named association, it might be written as follows:

```
Another_Procedure(Formal_2 => Actual_2, Formal_1 => Actual_1,
    Formal_3 => Actual_3);
```

We have intentionally rearranged the parameter order to illustrate the fact that it is possible to use any order when named association is used, not because it is a recommended practice. The situation in which named association has the greatest benefit is when the actual parameter is a literal constant. In such cases, the presence of a formal parameter name enhances the readability of the call. An alternative in languages without named association would be to adopt the style of always using named rather than literal constants.

Array Parameters

Some issues are particular to parameters that are arrays. To understand the first of these issues, we begin by discussing a problem that exists with Pascal array parameters. In Pascal, array type definitions are required to include the specific array bounds. Consequently, an array of 10 integers can never be of the same type as an array of 5 integers. Furthermore, Pascal requires that the type of the actual parameter and the formal parameter match. The consequence of these two features is that in Pascal, one cannot write a subprogram that processes arrays of different lengths. This is a serious language-design problem because it requires unnecessary code duplication.

Ada's solution to this problem was to introduce unconstrained array types. We saw an example of an unconstrained array type in module 2 as a part of our discussion of arrays, but let's consider another example.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
with Text_IO; use Text_IO;
procedure Main is
type IntegerArray is array (Integer range <>) of Integer;

    function SumArray (FormalArray: in IntegerArray) return Int
        Sum: Integer := 0;
    begin
        for I in FormalArray'Range loop
            Sum := Sum + FormalArray(I);
        end loop;
        return Sum;
    end SumArray;

    ActualArray_1: IntegerArray(1..5);
    ActualArray_2: IntegerArray(1..3);
begin
    ActualArray_1 := (1, 2, 3, 4, 5);
    Put(Integer'Image(SumArray(ActualArray_1)));
    ActualArray_2 := (1, 2, 3);
    Put(Integer'Image(SumArray(ActualArray_2)));
end Main;
```

Ada solves the problem with array parameters in Pascal by decoupling the specific array bounds from the array type.

Let's consider how this same problem would be written in C++. We examined a number of variations of a similar function in conjunction with pointer arithmetic in module 2, but let's now consider both the function and an accompanying main function.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int sum_array(int array[],int length);

int main()
{
    int actual_1[] = {1, 2, 3, 4, 5};
    int actual_2[] = {1, 2, 3};

    cout << sum_array(actual_1, sizeof(actual_1) / sizeof(int))
        << endl;
    cout << sum_array(actual_2, sizeof(actual_2) / sizeof(int))
        << endl;
    return 0;
}

int sum_array(int array[], int length)
{
    int sum = 0;

    for (int i = 0; i < length; i++)
        sum += array[i];
    return sum;
}
```

Although it is possible to write functions that process arrays of different lengths, the approach used in C and C++ can be the source of problems, as we noted in module 2 in our discussion of the C string functions. If the wrong length information is provided to the function—the worst case being a length greater than the actual length—memory outside the array bounds will be accessed. Recall that no bounds checking is done in these languages, so in functions that modify arrays, memory outside the array can be overwritten.

In case you think this problem could be remedied by eliminating the length parameter and replacing length in the for loop with sizeof(array) / sizeof(int), you must realize that sizeof is evaluated at compile time. There is no way at run time to determine the size of an array because no size information remains at run time.

Finally, let's look at the way this same program would be written in Java.

Move your mouse over the highlighted part of the program to see the explanation of that part.

```
class ArrayParameters
{
    public static void main(String[] args)
    {
        int[] actual_1 = {1, 2, 3, 4, 5};
        int[] actual_2 = {1, 2, 3};

        System.out.println(sumArray(actual_1));
        System.out.println(sumArray(actual_2));
    }
    private static int sumArray(int[] array)
    {
        int sum = 0;

        for (int i = 0; i < array.length; i++)
            sum += array[i];
        return sum;
    }
}
```

There is one final issue regarding array parameters that requires discussion—when parameters are multidimensional arrays. For programming languages such as C and C++ that do not store the array dimensions at run time, there is some difficulty in writing functions that process arrays with dimensions of any length. Consider the following attempt at a function prototype with a two-dimensional array parameter.

```
int sum_matrix(int matrix[][],int rows, int cols);
```

The above prototype will not compile. Although the first dimension can be omitted, the second and subsequent ones cannot. The reason the number of columns cannot be omitted is because the compiler needs this information to compute the location of array elements.

The number of columns is a part of this formula, but the number of rows is not. A solution to this problem in C and C++ is for the programmer to encode this formula, which the compiler would otherwise generate, and write the function as though it were taking a one-dimensional array as a parameter, as shown in the program below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

int sum_matrix(int matrix[], int rows, int cols);

int main()
{
    int actual[][3] = {{1, 2, 3}, {4, 5, 6}};

    cout <<  sum_matrix(actual[0], 2, 3) << endl;
    return 0;
}

int sum_matrix(int matrix[], int rows, int cols)
{
    int sum = 0;

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            sum += matrix[i * cols + j];
    return sum;
}
```

This problem with passing multidimensional arrays in C and C++ does not exist in Java or Ada because in both languages the sizes of the dimensions are stored at run time.

Parameter Type Checking

In module 2, we introduced the topic of parameter type checking and explained how the type of actual parameters must be checked against the type of formal parameters and how type coercion might be performed in languages that allow it when the types do not match, but are compatible.

Now, we consider languages in which such type checking is impossible in some cases and explore the consequences in those cases. Specifically, we refer to early versions of C. Independent compilation was allowed, but in the early versions, it was possible to call a function in another source file without providing a function prototype. Actually, a limited prototype that specified only the return type was required for functions that did not return an int. Without such prototypes, the compiler would be unable to generate correct object code for calls to such functions.

These prototypes were not required to specify the types of parameters, however. If the type of actual and formal parameters were the same, the program would execute properly. But if they were different, no coercion would be possible, and the resulting program would behave in an unpredictable fashion. This language design was especially troublesome during the maintenance phase of a program, in which the type of one of the function's parameters might change, and forgetting to make the appropriate change on every call produced logic errors that were difficult to find.

Because of the serious nature of the problems that can result from a lack of parameter type checking, the latest version of C and all versions of C++ require that a prototype be present for every function that is called.

Subprogram Parameters

Like recursion, in many ways, subprogram parameters properly belong to the domain of functional programming, not to imperative programming. When we discuss functional programming in module 5, we will see what an essential role subprogram parameters play.

Nonetheless, most imperative languages support this feature in some form. Pascal does. C and C++ allow function pointer parameters. Ada 83 required that generic subprograms be used for this purpose, but Ada 95 allows a more direct implementation of subprogram parameters. C# has a somewhat unique feature, called *delegates*, for this purpose. Java has no direct way to accomplish this task.

In Java, a method must be wrapped in a stateless object to achieve what amounts to passing it as a parameter. Consider the following Java example that illustrates creating a stateless object whose primary role is to pass a method as a parameter.

Move your mouse over the highlighted parts of the class to see the explanations of those parts.

```
class MethodParameters
{
    interface IntegerSequence
    {
        int sequence(int i);
    }

    public static void main(String[] args)
    {
        int sum = sumTerms(
            new IntegerSequence()
                {
                    public int sequence(int i)
                    {
                       return i;
                    }
                },
            5);
        System.out.println("Sum = " + sum);

    }

    private static int sumTerms(IntegerSequence f, int n)
    {
        int sum = 0;

        for (int i = 0; i < n; i++)
            sum += f.sequence(i);
        return sum;
    }
}
```

There are two important design issues that any languages that allow subprogram parameters must consider. The first is the degree to which the parameters are type checked. Without some type checking, serious errors can occur at run time. If type checking is performed, the next consideration is how closely the signatures must match.

The other design consideration is what referencing environment to give to the formal subprogram parameter. Which of the following should it be?

1. the environment of the call to the formal subprogram parameter
2. the environment of the actual subprogram parameter definition
3. the environment of the call that passed the subprogram parameter

One final consideration is whether functions can return other subprograms as their return values. Compiled languages generally do not allow this capability, although C and C++ do allow pointers to functions to be returned. Interpreted languages more often have this ability. JavaScript, for example, allows newly created subprograms to be returned. This ability is extremely important in functional programming, so we will return to it in module 5.

C. Other Issues

There are two other issues pertaining to subprograms, but not specifically parameters, that we need to consider. They are overloading and macros. We will discuss each of these below.

Overloading

We have seen some examples of overloaded subprograms earlier in this module, but now we plan to define *overloading* and explore the extent to which various languages support it.

A language allows subprogram overloading if distinguishable subprograms of the same name are permitted. What constitutes *distinguishable* differs somewhat among languages. In C++ and Java, the parameters and their types must be different. In Ada, the return type of functions can also be used to distinguish overloaded functions.

Overloading is sometimes referred to as a kind of polymorphism, but it is very different from the polymorphism associated with inheritance. The key difference is that overload resolution can be done at compile time, whereas the resolution of true polymorphism must occur at run time. We will explore this latter kind of polymorphism in module 4 in conjunction with our discussion of object-orientation.

Among languages that allow subprogram overloading, there is variation as to whether they allow operators to be defined as functions and thus allow user-defined operator overloading. Ada, C++, and C# allow such operator overloading, but Java does not.

By allowing user-defined operator overloading, it is possible to create new data types in a language that appear to be built-in data types. To illustrate this point, let's consider the definition of a new data type for complex numbers defined in C++. (Actually, the C++ standard template library contains a class very similar to this one.) First, we examine the class definition that would be placed in a file complex.h.

Move your mouse over the highlighted parts of the class definition to see the explanations of those parts.

```
class Complex
{
public:
    Complex(double real, double imaginary);
    friend istream& operator>>(istream& in, Complex& complex);
    friend ostream& operator<<(ostream& out, Complex complex);
    friend Complex operator+(Complex left, Complex right);
    friend Complex operator-(Complex left, Complex right);
    friend Complex operator*(Complex left, Complex right);
private:
    double real, imaginary;
};
```

Unlike Java, the bodies of the member bodies of a class are defined separately. Those function bodies would be in a file called complex.cpp, shown below.

```
#include <iostream>
using namespace std;

#include "complex.h"

Complex::Complex(double real, double imaginary)
{
    this->real = real;
    this->imaginary = imaginary;
}

istream& operator>>(istream& in, Complex& complex)
{
    char i;
    in >> complex.real >> complex.imaginary >> i;
    return in;
}

ostream& operator<<(ostream& out, Complex complex)
{
    if (complex.imaginary >= 0)
        out << complex.real << " + " << complex.imaginary << "
    else
        out << complex.real << " - " << -complex.imaginary << '
    return out;
}

Complex operator+(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real + right.real;
    imaginaryResult = left.imaginary + right.imaginary;
    return Complex(realResult, imaginaryResult);
}
```

```
Complex operator-(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real - right.real;
    imaginaryResult = left.imaginary - right.imaginary;
    return Complex(realResult, imaginaryResult);
}

Complex operator*(Complex left, Complex right)
{
    double realResult, imaginaryResult;
    realResult = left.real * right.real -
        left.imaginary * right.imaginary;
    imaginaryResult = left.real * right.imaginary +
        left.imaginary * right.real;
    return Complex(realResult, imaginaryResult);
}
```

Critics of user-defined operator overloading claim that the misuse of this feature can lead to programs that are hard to read.

Macros

Although macros are technically not subprograms, they play a role similar to that of subprograms, so it is appropriate to discuss them now. Macros are most commonly found in assembly language. But C, which was intended to be used as a language to replace assembly, and consequently C++, both allow macros. In these languages, they are translated by the preprocessor, which makes a preliminary pass over the source code. Java dispensed with the preprocessor, so it also dropped the possibility of macros.

In our earlier discussion of the call-by-name parameter-passing technique, we mentioned that the macros use a similar technique—performing a textual substitution of the formal parameters with the actual parameters. The primary

difference is that with macros, this substitution is done at compile time. Consider the following C or C++ macro:

```
#define SQUARE(x) x * x
```

Notice that, like functions, macros have parameters and a body. The body is generally much shorter than the average function body, consisting most often of a single expression rather than a sequence of statements. There are tradeoffs between the use of macros and functions. Using macros can make a program execute faster because the overhead of function calls is eliminated, but they can enlarge programs because the body of the macro replaces every macro call. It is because of this tradeoff that macros tend to be short.

Programmers who use textual substitution must be aware of several problems. For example, consider the following invocation of the previously defined macro.

```
y = SQUARE(z + 1)
```

The code that will result from the textual substitution is the following:

```
y = z + 1 * z + 1;
```

Because of precedence, the result is $2z + 1$, not the intended $(z + 1)^2$. This problem can be avoided by adding parentheses around both instances of x in the macro definition as follows:

```
#define SQUARE(x) (x) * (x)
```

Actual parameters that have side effects can present another problem. Suppose we invoke our macro as follows:

```
z = 2;
y = SQUARE(++z);
```

Because two substitutions will be performed, z will be incremented twice, not once, as was undoubtedly intended.

D. Implementing Subprograms

The complexity of implementing subprograms increases as we move from languages that do not support recursion, to ones that do, and, finally, to those that allow nested subprograms.

Languages without Recursion

When languages do not allow recursion, like early versions of Fortran, static data can be used exclusively. In such languages, it is relatively simple to implement the linkage between the calling and called subprogram. The only real complexity is considering that a subprogram may be called from possibly many different places. Therefore, at the point of the call, it is necessary to save the return address so that once that called subprogram completes, it knows where to return.

Languages with Recursion

As we discussed in module 2, languages that support recursion must allocate local data as stack-dynamic data because there can be multiple activations of such subprograms. Each time a recursive subprogram is called, another instance of its activation record must be pushed onto the compiler's run-time stack. That activation record must contain the return address, the parameters passed to the subprogram, space for local variables, and space for any temporary variables. For function calls, the returned value is also placed on the stack. It is customary to also include a pointer to the top of the activation record of the calling subprogram, called a *dynamic link*, to simplify popping the activation record instance when the subprogram completes.

Languages That Allow Nested Subprograms

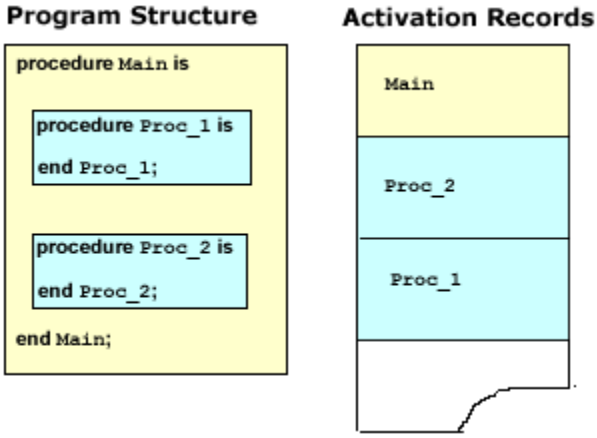
The greatest complexity in implementing the subprogram linkage is in languages that allow nested subprograms. Recall that none of the languages in the C family allow such nesting, but Pascal and Ada do. In module 2, in our discussion of stack-dynamic data, we mentioned that the compilers for such languages must determine two kinds of variable address information:

1. the relative address in the activation record
2. which activation record the variable is in

The latter value is referred to as the *static depth*. As we mentioned in module 2, this information alone is insufficient to locate the activation record at run time.

Figure 3.5 illustrates an Ada procedure containing two nested procedures, and the corresponding run-time stack of the compiler assuming that when the program runs, MAIN called PROC2 and PROC2 called PROC1.

Figure 3.5
An Ada Program with Nested Procedures
and the Compiler's Run-Time Stack



According to the scope rules of Ada, Proc_1 can access the local variables of Main but not Proc_2. Because Proc_1 is nested directly in Main, Main is considered to have a static depth of 1 away from Proc_1. The problem, however, is that the activation record for Main is two records away on the stack. Following and counting the dynamic links does not lead to the correct activation record.

There are two common solutions to this problem. The most common is to include what are called *static links* between activation records. The static link in the activation record for Proc_1 would point to the activation record for Main. In this way, following the static links and counting until the static depth was reached would lead to the correct record.

The other solution is to use what are called *displays*. A display is an array of pointers to activation records. In this example, when Proc_1 was executing, the pointer in position 0 of the display would point to the activation record for Proc_1, and the pointer in position 1 would point to the activation record for Main. So, the static depth would be used as an index into the display to find the correct activation record.

Languages that Allow Nested Blocks

All the languages in the C family allow nested blocks despite the fact that they do not allow nested subprograms. Nested blocks do not have any of the complexities of nested subprograms because blocks are not callable, and so can be encountered only when they are reached during program execution.